

Python 程式語言

Lecture 5: File Handling and Exception Handling

Instructor: 顏家珩 Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
 - Introduction to Python and Development Environment Setup
 - Basic Syntax and Flow Control
 - Data Structures and Collections
- **Advanced Programming**
 - Modular and Functional Programming
 - File Handling and Exception Handling
 - Object-Oriented Programming
 - Algorithm Implementation in Python
 - Modules and Package Applications
- User Interface Development
 - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
 - Data Analysis Fundamentals
 - Scientific Computing Applications
 - Data Visualization
- AI & ML Applications
 - Introduction to Machine Learning with Applications
 - AI-Related Library in Python Applications

Outline

- Master file reading and writing operations
- Understand exception handling mechanisms

File Handling

Why File Handling?

- Benefits
 - Data Persistence
 - Data Sharing
 - Large Data Processing
 - Configuration Storage
 - Logging and Records

```
# Without file handling  
user_data = "John Doe, 25, john@example.com"  
# Data lost when program ends!  
  
# With file handling  
with open('user_data.txt', 'w') as file:  
    file.write("John Doe, 25, john@example.com")  
# Data saved permanently!
```

File Opening Modes

- Basic Modes

Mode	Description	Creates File	Overwrites
'r'	Read only	No	-
'w'	Write	Yes	Yes
'a'	Append	Yes	No
'x'	Exclusive creation	Yes	Error if exists

- Additional Modifiers

Modifier	Description
'b'	Binary mode
't'	Text mode (default)
'+'	Read and write

Examples

```
file1 = open('read.txt', 'r') # Read text file
file2 = open('write.txt', 'w') # Write text file
file3 = open('append.txt', 'a') # Append to file
file4 = open('data.bin', 'rb') # Read binary file
file5 = open('data.txt', 'r+') # Read and write
file6 = open('new.txt', 'x') # Create new file
```

Basic File Operations

- Writing to Files

Method 1: Manual file handling

```
file = open('output.txt', 'w')
file.write("Hello, World!\n")
file.write("This is a test file.\n")
file.close() # Important! Must close file
```

Method 2: Using with statement (recommended)

```
with open('output.txt', 'w') as file:
    file.write("Hello, World!\n")
    file.write("This is a test file.\n")
```

File automatically closed

Writing multiple lines

```
lines = [
    "Line 1\n",
    "Line 2\n",
    "Line 3\n"
]

with open('output.txt', 'w') as file:
    file.writelines(lines) # Write list of strings
```

Appending to file

```
with open('output.txt', 'a') as file:
    file.write("This is appended.\n")
```

Basic File Operations

- Reading from Files

Read entire file

```
with open('input.txt', 'r') as file:  
    content = file.read()  
    print(content)
```

Read file line by line

```
with open('input.txt', 'r') as file:  
    for line in file:  
        print(line.strip()) # strip() removes newline
```

Read lines into list

```
with open('input.txt', 'r') as file:  
    lines = file.readlines()  
    print(f"Total lines: {len(lines)}")
```

Read specific number of characters

```
with open('input.txt', 'r') as file:  
    chunk = file.read(100) # Read first 100 characters  
    print(chunk)
```

Read one line at a time

```
with open('input.txt', 'r') as file:  
    line1 = file.readline() # First line  
    line2 = file.readline() # Second line  
    print(f"Line 1: {line1}")  
    print(f"Line 2: {line2}")
```


File Path Handling

- Working with Paths

```
import os

# Get current working directory
current_dir = os.getcwd()
print(f"Current directory: {current_dir}")

# Change directory
# os.chdir('/path/to/directory')

# Check if file exists
if os.path.exists('data.txt'):
    print("File exists!")
else:
    print("File does not exist!")

# Check if path is file or directory
path = 'example.txt'
print(f"Is file: {os.path.isfile(path)}")
print(f"Is directory: {os.path.isdir(path)}")
```

```
# Get file size
if os.path.exists('data.txt'):
    size = os.path.getsize('data.txt')
    print(f"File size: {size} bytes")

# Join paths (cross-platform)
file_path = os.path.join('folder', 'subfolder', 'file.txt')
print(f"Path: {file_path}")

# Split path into directory and filename
directory, filename = os.path.split(file_path)
print(f"Directory: {directory}")
print(f"Filename: {filename}")

# Get file extension
name, extension = os.path.splitext(filename)
print(f"Name: {name}, Extension: {extension}")
```

File Path Handling

- Using pathlib (Modern Approach)

```
from pathlib import Path

# Create Path object
file_path = Path('data/input.txt')

# Check if exists
if file_path.exists():
    print("File exists!")

# Get file info
if file_path.is_file():
    print(f"File name: {file_path.name}")
    print(f"File stem: {file_path.stem}")
    print(f"File suffix: {file_path.suffix}")
    print(f"Parent directory: {file_path.parent}")
```

```
# Read file using Path
content = file_path.read_text()
print(content)

# Write file using Path
file_path.write_text("Hello from pathlib!")

# List files in directory
directory = Path('.')
for item in directory.iterdir():
    if item.is_file():
        print(f"File: {item.name}")
```

CSV File Handling

- Reading CSV Files

```
import csv

# Reading CSV with csv.reader
with open('data.csv', 'r', encoding='utf-8') as file:
    csv_reader = csv.reader(file)

    # Read header
    header = next(csv_reader)
    print(f"Header: {header}")

    # Read data rows
    for row in csv_reader:
        print(row)
```

```
# Reading CSV with DictReader
with open('students.csv', 'r', encoding='utf-8') as file:
    csv_reader = csv.DictReader(file)

    for row in csv_reader:
        print(f"Name: {row['name']], Age: {row['age']],
        Grade: {row['grade']}")

# Handling different delimiters
with open('data.tsv', 'r', encoding='utf-8') as file:
    csv_reader = csv.reader(file, delimiter='\t') # Tab-separated values

    for row in csv_reader:
        print(row)

# Example CSV content: students.csv
# name,age,grade
# Alice,20,A # Bob,22,B # Carol,21,A
```

CSV File Handling

- Writing CSV Files

```
import csv

# Writing with csv.writer
data = [
    ['Name', 'Age', 'City'],
    ['Alice', '25', 'Taipei'],
    ['Bob', '30', 'Kaohsiung'],
    ['Carol', '28', 'Taichung']
]

with open('output.csv', 'w', newline="", encoding='utf-8') as file:
    csv_writer = csv.writer(file)
    csv_writer.writerows(data) # Write all rows
```

```
# Writing with DictWriter
students = [
    {'name': 'Alice', 'age': 20, 'grade': 'A'},
    {'name': 'Bob', 'age': 22, 'grade': 'B'},
    {'name': 'Carol', 'age': 21, 'grade': 'A'}
]

with open('students_output.csv', 'w', newline="", encoding='utf-8') as file:
    fieldnames = ['name', 'age', 'grade']
    csv_writer = csv.DictWriter(file, fieldnames=fieldnames)

    csv_writer.writeheader() # Write header
    csv_writer.writerows(students) # Write data

# Appending to CSV
with open('students_output.csv', 'a', newline="", encoding='utf-8') as file:
    csv_writer = csv.writer(file)
    csv_writer.writerow(['David', '23', 'B'])
```

JSON File Handling

- Reading JSON Files

```
import json

# Read JSON file
with open('data.json', 'r', encoding='utf-8') as file:
    data = json.load(file) # Parse JSON into Python object
    print(data)
    print(f"Type: {type(data)}")

# Example JSON content: data.json
# {
#     "name": "Alice",
#     "age": 25,
#     "city": "Taipei",
#     "hobbies": ["reading", "coding", "gaming"]
# }

# Accessing JSON data
print(f"Name: {data['name']}")
print(f"Age: {data['age']}")
print(f"Hobbies: {', '.join(data['hobbies'])}")
```

```
# Reading JSON array
with open('users.json', 'r', encoding='utf-8') as file:
    users = json.load(file)
    for user in users:
        print(f"User: {user['name']}, Email: {user['email']}")

# Example JSON array: users.json
# [
#     {"name": "Alice", "email": "alice@example.com"},
#     {"name": "Bob", "email": "bob@example.com"}
# ]

# Parsing JSON string
json_string = '{"status": "success", "code": 200}'
data = json.loads(json_string) # Note: loads() not load()
print(data)
```

JSON File Handling

- Writing JSON Files

Write Python object to JSON

```
user_data = {  
    'name': 'Alice',  
    'age': 25,  
    'city': 'Taipei',  
    'hobbies': ['reading', 'coding', 'gaming'],  
    'is_student': True,  
    'score': 95.5  
}
```

```
with open('user_data.json', 'w', encoding='utf-8') as file:  
    json.dump(user_data, file, indent=4, ensure_ascii=False)
```

indent=4: Pretty print with 4-space indentation

ensure_ascii=False: Allow Unicode characters

Write multiple records

```
users = [  
    {'name': 'Alice', 'age': 25, 'city': 'Taipei'},  
    {'name': 'Bob', 'age': 30, 'city': 'Kaohsiung'},  
    {'name': 'Carol', 'age': 28, 'city': 'Taichung'}  
]
```

```
with open('users.json', 'w', encoding='utf-8') as file:  
    json.dump(users, file, indent=2, ensure_ascii=False)
```

Convert to JSON string

```
json_string = json.dumps(user_data, indent=2, ensure_ascii=False)  
print(json_string)  
print(f"Type: {type(json_string)}")
```

Update JSON file

```
with open('config.json', 'r', encoding='utf-8') as file:  
    config = json.load(file)
```

Modify data

```
config['version'] = '2.0'  
config['last_updated'] = '2025-10-20'
```

```
with open('config.json', 'w', encoding='utf-8') as file:  
    json.dump(config, file, indent=4, ensure_ascii=False)
```

The with Statement

- Why use with?

Without with statement

```
file = open('data.txt', 'r')
```

```
try:
```

```
    content = file.read()
```

```
    print(content)
```

```
finally:
```

```
    file.close() # Must manually close
```

With statement (recommended)

```
with open('data.txt', 'r') as file:
```

```
    content = file.read()
```

```
    print(content)
```

File automatically closed, even if exception occurs

Benefits of with statement

1. Automatic resource cleanup

2. Cleaner code

3. Exception-safe

The with Statement

- Multiple Files with with

```
# Open multiple files
with open('input.txt', 'r') as infile, open('output.txt', 'w') as outfile:
    content = infile.read()
outfile.write(content.upper()) # Convert to uppercase

# Copy file line by line
with open('source.txt', 'r') as source, open('destination.txt', 'w') as dest:
    for line in source:
        dest.write(line)
```

```
# Process multiple files
file_names = ['file1.txt', 'file2.txt', 'file3.txt']

for file_name in file_names:
    try:
        with open(file_name, 'r') as file:
            content = file.read()
            print(f"Content of {file_name}:")
            print(content)
            print("-" * 40)
    except FileNotFoundError:
        print(f"File {file_name} not found!")
```


Exception Handling

What are Exceptions?

- Definition
 - An **exception** is an error that occurs during program execution
- Common Built-in Exceptions

1. ZeroDivisionError

```
result = 10 / 0 # Error!
```

2. ValueError

```
number = int("abc") # Error!
```

3. TypeError

```
result = "5" + 5 # Error!
```

4. FileNotFoundError

```
with open('nonexistent.txt', 'r') as file:  
    content = file.read() # Error!
```

5. IndexError

```
my_list = [1, 2, 3]  
item = my_list[10] # Error!
```

6. KeyError

```
my_dict = {'a': 1, 'b': 2}  
value = my_dict['c'] # Error!
```

7. AttributeError

```
my_string = "hello"  
my_string.append('!') # Error! Strings don't have append()
```

8. ImportError

```
import nonexistent_module # Error! 錯誤!
```

try-except Structure

- Basic Syntax

```
try:
    # Code that might raise an exception
    risky_operation()
except ExceptionType:
    # Code to handle the exception
    handle_error()
```

- Simple Example

```
# Without exception handling
number = int(input("Enter a number: ")) # Crashes if invalid input!
print(f"You entered: {number}")
```

```
# With exception handling
try:
    number = int(input("Enter a number: "))
    print(f"You entered: {number}")
except ValueError:
    print("Invalid input! Please enter a valid number.")
```

```
# Division with exception handling
def safe_divide(a, b):
    """Safely divide two numbers"""
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")
        return None

# Test the function
print(safe_divide(10, 2)) # 5.0
print(safe_divide(10, 0)) # Error message, returns None
```

try-except Structure

- Catching Exception Details

Get exception information

try:

number = int(input("Enter a number: "))

result = 100 / number

except ValueError as e:

print(f"ValueError occurred: {e}")

except ZeroDivisionError as e:

print(f"ZeroDivisionError occurred: {e}")

Example with file operations

try:

with open('data.txt', 'r') as file:

content = file.read()

print(content)

except FileNotFoundError as e:

print(f"File error: {e}")

except PermissionError as e:

print(f"Permission denied: {e}")

Multiple Exception Handling

- Handling Different Exceptions

```
def process_data(filename, index):  
    """Process data with multiple exception handling"""  
    try:  
        # Open file  
        with open(filename, 'r') as file:  
            lines = file.readlines()  
  
        # Access specific line  
        line = lines[index]  
  
        # Convert to number  
        number = int(line.strip())  
  
        # Perform calculation  
        result = 100 / number  
  
    return result
```

```
except FileNotFoundError:  
    print(f"Error: File '{filename}' not found!")  
    return None  
  
except IndexError:  
    print(f"Error: Index {index} out of range!")  
    return None  
  
except ValueError:  
    print(f"Error: Line contains non-numeric data!")  
    return None  
  
except ZeroDivisionError:  
    print(f"Error: Cannot divide by zero!")  
    return None  
  
# Test the function  
result = process_data('numbers.txt', 0)  
print(f"Result: {result}")
```

Multiple Exception Handling

- Catching Multiple Exceptions Together

Method 1: Tuple of exceptions

```
try:
    # Some risky code
    value = int(input("Enter a number: "))
    result = 10 / value
except (ValueError, ZeroDivisionError) as e:
    print(f"Input or calculation error: {e}")
```

Method 2: Separate except blocks

```
try:
    data = {'name': 'Alice', 'age': 25}
    print(data['email']) # KeyError
    result = 10 / 0 # ZeroDivisionError
except KeyError:
    print("Key not found in dictionary!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Catch all exceptions

```
try:
    # Some code
    risky_operation()
except Exception as e:
    print(f"An error occurred: {type(e).__name__}: {e}")
```

else and finally Clauses

- else Clause

```
# else: Executed if no exception occurs
try:
    number = int(input("Enter a number: "))
    result = 100 / number
except ValueError:
    print("Invalid input!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    # This runs only if no exception occurred
    print(f"Calculation successful! Result: {result}")
    print("No errors encountered.")
```

```
# Practical example: File processing
def read_and_process_file(filename):
    """Read file with else clause"""
    try:
        with open(filename, 'r') as file:
            content = file.read()
    except FileNotFoundError:
        print(f"File '{filename}' not found!")
        return None
    else:
        # Process content only if file was read successfully
        print("File read successfully!")
        lines = content.split("\n")
        print(f"Total lines: {len(lines)}")
        return content
```

else and finally Clauses

- finally Clause

finally: Always executed, regardless of exceptions

try:

file = open('data.txt', 'r')

content = file.read()

print(content)

except FileNotFoundError:

print("File not found!")

finally:

This always runs

print("Cleaning up...")

try:

file.close()

print("File closed successfully")

except:

print("File was not opened")

Complete example with try-except-else-finally

def divide_numbers(a, b):

"""Complete exception handling example"""

result = None

try:

print(f"Attempting to divide {a} by {b}")

result = a / b

except ZeroDivisionError:

print("Error: Division by zero!")

except TypeError:

print("Error: Invalid types for division!")

else:

print(f"Division successful! Result: {result}")

return result

finally:

print("Division operation completed.")

print("-" * 40)

return None

Test cases

divide_numbers(10, 2) # Success

divide_numbers(10, 0) # ZeroDivisionError

divide_numbers(10, "abc") # TypeError

Custom Exceptions

- Creating Custom Exceptions

Basic custom exception

```
class MyCustomError(Exception):
    """Custom exception class"""
    pass

# Using custom exception
def check_age(age):
    """Check if age is valid"""
    if age < 0:
        raise MyCustomError("Age cannot be negative!")
    if age > 150:
        raise MyCustomError("Age seems unrealistic!")
    print(f"Age {age} is valid.")

# Test custom exception
try:
    check_age(25) # Valid
    check_age(-5) # Raises MyCustomError
except MyCustomError as e:
    print(f"Custom error: {e}")
```

Custom exception with additional attributes

```
class ValidationError(Exception):
    """Exception for validation errors"""
    def __init__(self, message, field_name=None, value=None):
        self.message = message
        self.field_name = field_name
        self.value = value
        super().__init__(self.message)
    def __str__(self):
        if self.field_name:
            return f"ValidationError in '{self.field_name}': {self.message} (value: {self.value})"
        return f"ValidationError: {self.message}"

# Using ValidationError
def validate_user_data(username, age):
    """Validate user data"""
    if not username:
        raise ValidationError("Username cannot be empty!", "username", username)
    if len(username) < 3:
        raise ValidationError("Username too short!", "username", username)
    if age < 18:
        raise ValidationError("Must be 18 or older!", "age", age)
    print("Validation passed!")

# Test validation
try:
    validate_user_data("Jo", 25)
except ValidationError as e:
    print(e)

try:
    validate_user_data("John", 16)
except ValidationError as e:
    print(e)
```

Custom Exceptions

- Exception Hierarchy

```
# Creating exception hierarchy
class DatabaseError(Exception):
    """Base exception for database errors"""
    pass

class ConnectionError(DatabaseError):
    """Exception for connection errors"""
    pass

class QueryError(DatabaseError):
    """Exception for query errors"""
    pass

class DataNotFoundError(DatabaseError):
    """Exception when data is not found"""
    pass
```

```
# Using the hierarchy
def execute_query(query):
    """Execute database query"""
    if not query:
        raise QueryError("Query cannot be empty!")
    if query == "SELECT * FROM nonexistent":
        raise DataNotFoundError("Table does not exist!")
    print(f"Executing query: {query}")

# Catch specific or general exceptions
try:
    execute_query("")
except DataNotFoundError as e:
    print(f"Data error: {e}")
except QueryError as e:
    print(f"Query error: {e}")
except DatabaseError as e:
    # Catches any DatabaseError subclass
    print(f"Database error: {e}")
```

assert Statement

- What is assert?

```
# assert checks if condition is True  
# If False, raises AssertionError  
  
# Basic syntax  
assert condition, "error message"  
  
# Examples  
age = 25  
assert age >= 18, "Age must be 18 or older"  
print("Age check passed!")  
  
# Without message  
x = 10  
assert x > 0 # Passes silently  
assert x < 5 # AssertionError!
```

assert Statement

- Using assert for Debugging

```
def calculate_average(numbers):  
    """Calculate average with assertions"""  
    # Pre-conditions  
    assert len(numbers) > 0, "List cannot be empty"  
    assert all(isinstance(n, (int, float)) for n in numbers), "All  
elements must be numbers"  
  
    average = sum(numbers) / len(numbers)  
  
    # Post-conditions  
    assert isinstance(average, (int, float)), "Average must be a  
number"  
  
    return average  
  
# Test assertions  
try:  
    result = calculate_average([1, 2, 3, 4, 5])  
    print(f"Average: {result}")  
  
    # This will fail  
    result = calculate_average([])  
except AssertionError as e:  
    print(f"Assertion failed: {e}")
```

```
# Assert in function validation  
def divide(a, b):  
    """Divide with assertions"""  
    assert isinstance(a, (int, float)), "First argument must be a  
number"  
    assert isinstance(b, (int, float)), "Second argument must be a  
number"  
    assert b != 0, "Cannot divide by zero"  
  
    return a / b  
  
# Test divide function  
try:  
    print(divide(10, 2)) # 5.0  
    print(divide(10, 0)) # AssertionError  
except AssertionError as e:  
    print(f"Error: {e}")
```

assert Statement

- When to Use assert

✓ Good uses of assert

1. Testing invariants

```
def set_age(person, age):  
    assert 0 <= age <= 150, "Age must be between 0 and 150"  
    person['age'] = age
```

2. Checking function inputs/outputs

```
def get_positive_sqrt(number):  
    assert number >= 0, "Number must be non-negative"  
    result = number ** 0.5  
    assert result >= 0, "Square root must be non-negative"  
    return result
```

3. Development and debugging

```
DEBUG = True
```

```
if DEBUG:
```

```
    assert user_id is not None, "User ID should be set by now"
```

✗ Bad uses of assert

Don't use for user input validation

BAD: assert user_input.isdigit(), "Enter a valid number"

GOOD: Use try-except or if statements instead

Don't use for data validation

BAD: assert file_exists, "File must exist"

GOOD: Use proper exception handling

Note: Assertions can be disabled with -O flag

python -O script.py # Assertions are ignored!